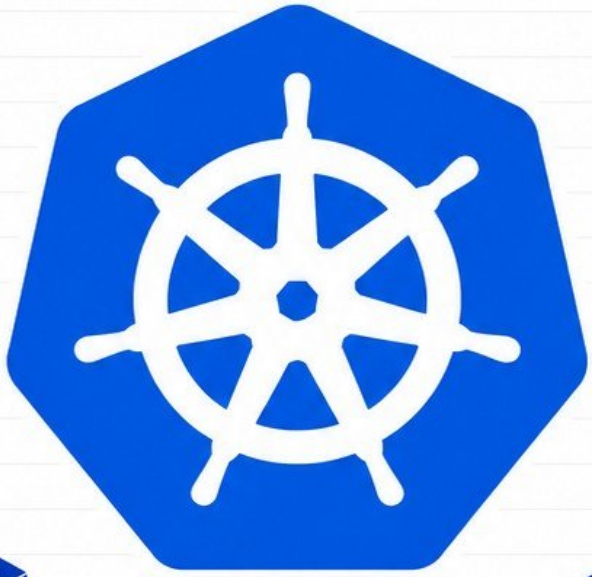
VERIQTA

# KUBERNETES

# HOME LAB HANDBOOK

BUILD • DEPLOY • OBSERVE • SECURE



BUILD YOUR CLUSTER

DEPLOY REAL WORKLOADS

MONITOR & OBSERVE

SECURE YOUR ENVIRONMENT

PRACTICE LIKE PRODUCTION

**PRACTICAL**
Step-by-step lab exercises for real hands-on learning

**PRODUCTION-FOCUSED**
Best practices, architecture patterns, and real-world scenarios

**TROUBLESHOOTING**
Debug issues, understand systems, and build deep Kubernetes skills

YOUTUBE /veriqta | GITHUB /veriqta | LINKEDIN /veriqta | X /veriqta | INSTAGRAM /veriqta | TELEGRAM /veriqta

# 1. HOME LAB ARCHITECTURE

## WHAT THE LAB WILL SIMULATE

- A production-like Kubernetes environment running on your local network.
- High availability control plane.
- Multiple worker nodes for real workloads.
- Networking, storage, and observability stack.
- Failure scenarios and recovery.

## CONTROL PLANE & WORKER NODES

### Control Plane (3 nodes)

- kube-apiserver, etcd, scheduler, controller-manager
- Provides cluster management and decisions
- HA setup for reliability

### Worker Nodes (3 nodes)

- Run your containerized applications
- Host system add-ons and monitoring agents

## MANAGEMENT WORKSTATION

- Your admin machine (laptop/PC)
- Runs kubectl, Helm, and tools
- SSH access to all nodes
- Not a cluster node
- Used for management only

## NETWORK TOPOLOGY

- All nodes on a dedicated private network
- Full mesh connectivity
- Pods communicate through CNI overlay network
- Services exposed via Ingress or LoadBalancer (MetalLB)

## IP ADDRESS PLANNING

Example IP Plan (Lab Network: 192.168.50.0/24)

| | |
|---|---|
| Control Plane 1 | 192.168.50.11 |
| Control Plane 2 | 192.168.50.12 |
| Control Plane 3 | 192.168.50.13 |
| Worker Node 1 | 192.168.50.21 |
| Worker Node 2 | 192.168.50.22 |
| Worker Node 3 | 192.168.50.23 |
| Management PC | 192.168.50.100 |
| VIP (API Load Balancer) | 192.168.50.10 |

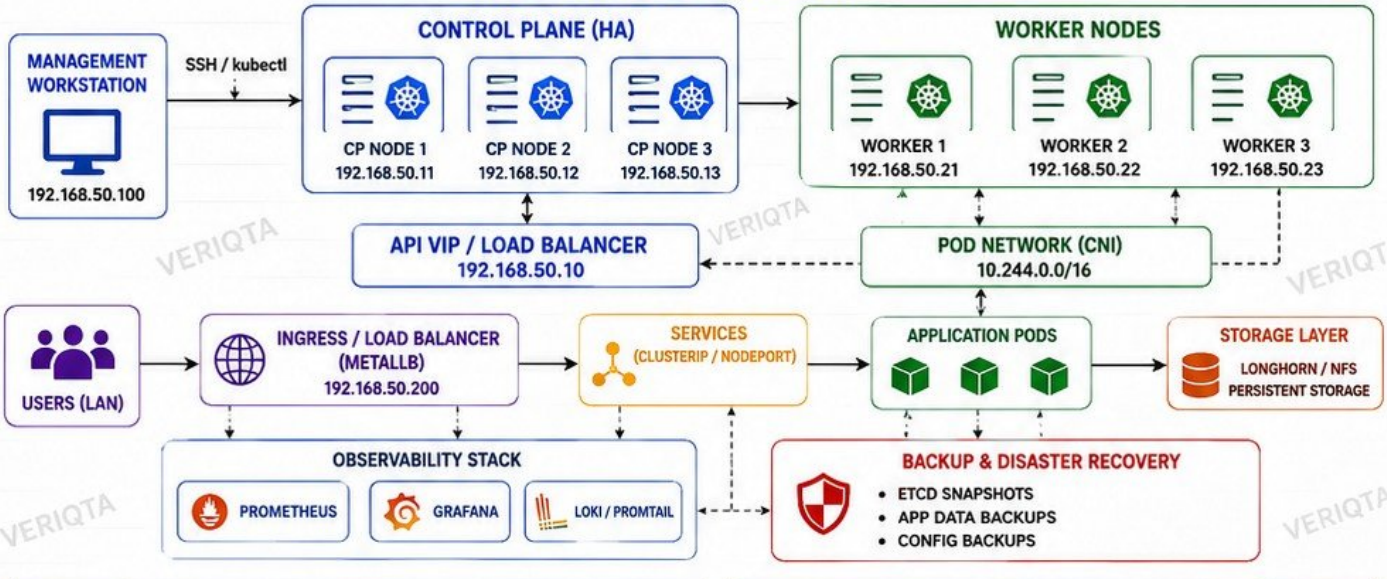
## STORAGE DESIGN

- etcd: Local SSD on control plane nodes
- Persistent Data: Longhorn (distributed block storage) or NFS server on separate host
- Backups: Automated snapshots to external drive/NAS
- Storage Classes for dynamic provisioning

## PRODUCTION CONCEPTS THIS LAB WILL REPRODUCE

- High Availability Control Plane
- Rolling Updates & Rollbacks
- Auto Healing & Self Recovery
- Networking (CNI, Ingress, DNS)
- Persistent Storage
- Monitoring & Logging
- RBAC & Security Best Practices
- Backup & Disaster Recovery

## FINAL ARCHITECTURE DIAGRAM




**NOTE:** This architecture is scalable. Add more worker nodes, change storage, and extend networking as your lab grows.

**TIP:** Start small, learn deeply, then scale your lab as you grow.

# 2. HARDWARE AND LAB DESIGN DECISIONS

## PHYSICAL MACHINES VERSUS VMS

| PHYSICAL MACHINES | VIRTUAL MACHINES |
|---|---|
| Better performance | Lower cost to start |
| Full hardware access | Quick to create and destroy |
| Lower latency | Easy to snapshot/backup |
| More reliable for labs | Flexible resource allocation |
| Higher upfront cost | Small performance overhead |
| Less flexible to scale | Requires host resources |

## MINIMUM PRACTICAL RESOURCES

| NODE TYPE | CPU (MIN) | RAM (MIN) | DISK (MIN) | ROLE |
|---|---|---|---|---|
| CONTROL PLANE | 4 vCPU | 8 GB | 50 GB SSD | etcd, API, Sched, Ctrl Mgr |
| WORKER NODE | 4 vCPU | 8 GB | 50 GB SSD | Run workloads |
| MANAGEMENT PC | 4 vCPU | 8 GB | 50 GB SSD | kubectl, tools, access |

**RECOMMENDATION:**
6 vCPU and 16 GB RAM per node is ideal for comfort and future growth.

## CPU, RAM, DISK, AND NETWORK PLANNING

- **CPU:** More cores = better scheduling and parallel workloads
- **RAM:** Kubernetes and etcd need memory to stay stable
- **DISK:** Use SSD for control plane and workloads; fast I/O is critical
- **NETWORK:** Gigabit network minimum; 10GbE recommended for larger labs

**TIP:**
Slow disks and low RAM cause the most Kubernetes problems.

## BARE METAL VERSUS VIRTUALIZATION

### BARE METAL

- Maximum performance and stability
- Best for production-like behavior
- Recommended for final lab setup

### VIRTUALIZATION

- Great for learning and testing
- Snapshots and cloning are easy
- Use type-1 hypervisors (Proxmox, ESXi, Hyper-V) for best results

**BEST PRACTICE:**
Start with VMs, then move critical components to bare metal.

## RESOURCE ALLOCATION PER NODE (SUGGESTED)

| NODE TYPE | CPU | RAM | DISK |
|---|---|---|---|
| CONTROL PLANE | 4–6 vCPU | 8–16 GB | 50–100 GB |
| WORKER NODE | 4–8 vCPU | 8–16 GB | 50–200 GB |
| MANAGEMENT PC | 4+ vCPU | 8–16 GB | 50+ GB |

**NOTE:**
Adjust based on the workloads you plan to run in your lab.

## DESIGNING FOR FUTURE EXPANSION

- Leave CPU, RAM, and disk headroom on every node.
- Use scalable storage solutions (NFS, Ceph, or longhorn).
- Plan your IP ranges with extra space.
- Design your network and VLANs for additional nodes.
- Keep your architecture modular and documented.

**PLAN TODAY, SCALE TOMORROW.**
A well-designed lab grows with your skills.

## AVOIDING AN UNDERPOWERED CONTROL PLANE

- etcd needs RAM and fast storage.
- Low CPU causes API server timeouts.
- Underpowered control plane = cluster instability.
- Never run control plane on the same node as heavy workloads.
- Allocate dedicated resources only for control plane.

**WARNING:**
Most Kubernetes issues in labs come from a weak control plane.

## KEY TAKEAWAY

Good hardware decisions create a stable, high-performing, and scalable Kubernetes lab.

RIGHT HARDWARE = STABILITY | RIGHT DESIGN = RELIABILITY | RIGHT PLAN = GROWTH | RIGHT CONTROL PLANE = SUCCESS

# 3. BUILDING THE LINUX NODES

## 1 UBUNTU SERVER PREPARATION

- Install Ubuntu Server LTS (22.04 recommended).
- During install: OpenSSH Server.
- Update the system.
- Create a non-root user with sudo access.

```
sudo apt update && sudo apt upgrade -y
sudo adduser labadmin
sudo usermod -aG  sudo labadmin
sudo systemctl enable ssh --now
```

## 2 HOSTNAMES AND STATIC ADDRESSING

- Set a clear hostname for each node.
- Configure static IP address.
- Update /etc/hosts on all nodes.

| NODE | HOSTNAME | ROLE | IP ADDRESS |
|---|---|---|---|
| Control Plane 1 | cp1.lab.local | Control Plane | 192.168.50.11 |
| Worker Node 1 | worker1.lab.local | Worker | 192.168.50.21 |
| Worker Node 2 | worker2.lab.local | Worker | 192.168.50.22 |
| Worker Node 3 | worker3.lab.local | Worker | 192.168.50.23 |

**TIP:** Use a consistent naming and IP scheme. Update /etc/hosts on all nodes for name resolution.

## 3 SSH ACCESS

- Enable SSH on all nodes.
- Use key-based authentication.
- Disable root login.
- Test SSH from control plane to all nodes.

```
ssh-keygen -t ed25519 -C "lab"
ssh-copy-id labadmin@worker1.lab.local
ssh-copy-id labadmin@worker2.lab.local
ssh-copy-id labadmin@worker3.lab.local
ssh cp1.lab.local
ssh worker1.lab.local
ssh worker2.lab.local
ssh worker2.lab.local
```

## 4 TIME SYNCHRONIZATION

- Accurate time is critical for Kubernetes.
- Install and enable chrony (recommended).
- Verify time sync on all nodes.

```
sudo apt install -y chrony
sudo systemctl enable --now chrony
chronyc sources -v
chronyc tracking
```

## 5 KERNEL REQUIREMENTS

- Kubernetes requires specific kernel features to work properly.
- Ubuntu 22.04 LTS kernel is supported.
- Verify kernel version and modules.

```
uname -r
lsmod | egrep 'br_netfilter|overlay'
cat /proc/sys/net/bridge/bridge-nf-call-
iptables
cat /proc/sys/net/ipv4/ip_forward
# All should return 1
```

## 6 SWAP CONFIGURATION

- Disable swap (required by Kubernetes).
- Remove from /etc/fstab.
- Confirm swap is off.

```
sudo swapoff -a
sudo sed -i '/swap/d' /etc/fstab
free -h
# Swap: 0 or "off"
```

**WARNING:** Kubernetes will not start properly if swap is enabled.

## 7 REQUIRED PACKAGES

- Install essential tools on all nodes.
- These are needed for cluster setup and troubleshooting.

```
sudo apt update
sudo apt install -y \
 curl wget git vim htop net-tools \
 iproute2 iptables ca-certificates \
 gnupg lsb-release apt-transport-https
```

**NOTE:** Additional packages will be installed during Kubernetes setup.

## 8 NODE-TO-NODE CONNECTIVITY CHECKS

- All nodes must communicate on required ports.
- Test ping, DNS, and key Kubernetes ports.
- Required ports: 6443, 2379–2380, 10250, 10259, 10257, 30000–32767, etc.

```
ping worker1.lab.local
ping worker2.lab.local
ping worker3.lab.local
nc -zv cp1.lab.local 6443
nc -zv worker1.lab.local 10250
nc -zv worker2.lab.local 10250
nc -zv worker3.lab.local 10250
```

**TIP:** Fix any connectivity issue before proceeding to cluster setup.

## READY CHECK SUMMARY

- System updated
- Hostname and IP set
- SSH access working
- Time synchronized
- Kernel verified
- Swap disabled
- Required packages installed
- All nodes reachable

**YOU ARE READY FOR KUBERNETES SETUP!**

# 4. CONTAINER RUNTIME WITH CONTAINERD

containerd

## WHY KUBERNETES NEEDS A CRI RUNTIME

- Kubernetes communicates with the container runtime through the CRI (Container Runtime Interface).
- CRI provides a standard way to pull images, run containers, and manage their lifecycle.
- containerd is lightweight, fast, and production-tested.
- Supports Kubernetes features like Pods, logging, and exec.

## 2 INSTALLING CONTAINERD

- Update system and install prerequisites.
- Install containerd.
- Enable and start containerd service.
- Verify installation.

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/
ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install -y containerd.io
sudo systemctl enable --now containerd
sudo systemctl status containerd --no-pager
```

## 3 SYSTEMD CGROUPS

- Kubernetes requires systemd cgroup driver for stability and resource management.
- Configure containerd to use systemd as the cgroup driver.
- Restart containerd after changes.

```
sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee \
/etc/containerd/config.toml >/dev/null
# Edit the file:
# Set the following:
  SystemdCgroup = true
sudo systemctl restart containerd
```

**TIP:** Verify kubelet also uses systemd cgroup driver in /var/lib/kubelet/config.yaml

## 4 CONTAINERD CONFIGURATION

- Key settings for Kubernetes environments.
- Important sections in config.toml.
- Keep the config minimal and clean.

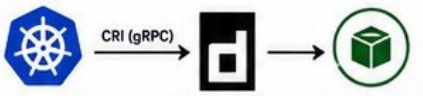
```
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.options]
  SystemdCgroup = true

[plugins."io.containerd.grpc.v1.cri".sandbox_image]
  registry.k8s.io/pause:3.9
```

**TIP:** Avoid changing advanced settings unless you understand the impact.

## 5 CRI INTEGRATION

- kubelet talks to containerd through CRI.
- Ensure the CRI socket is active.
- Default socket for containerd:

```
/run/containerd/containerd.sock
```




## 6 RUNTIME VERIFICATION

- Confirm containerd is running.
- Check version and CRI connectivity.
- Pull and run a test container.

```
sudo systemctl status containerd --no-pager
containerd  --version
ctr version
crictl version
crictl info | grep runtimeType
crictl pull registry.k8s.io/pause:3.9
crictl run registry.k8s.io/pause:3.9 testpause \
  sandbox-image registry.k8s.io/pause:3.9
crictl ps -a
```

## 7 COMMON CONTAINERD FAILURES AND FIXES

| ISSUE | SYMPTOM | CAUSE | FIX |
|---|---|---|---|
| Containerd not starting | Service fails to start | Config error or incompatible settings | Check: sudo journalctl -u containerd -xe<br>Validate config.toml and restart service |
| CRI socket not found | kubelet cannot connect | containerd not running or wrong socket path | Start containerd and verify socket: /run/containerd/containerd.sock |
| cgroup driver mismatch | Pods fail to start or restart in loop | kubelet and containerd using different cgroup drivers | Set SystemdCgroup = true<br>Ensure kubelet cgroupDriver: systemd |
| Image pull failures | Cannot pull images | Network issues, registry blocked, or DNS problems | Check DNS, internet, proxy, and registry access<br>Try: crictl pull registry.k8s.io/pause:3.9 |
| Out of disk space | Pod creation fails | Low disk on /var/lib/containerd | Clean up images/containers: crictl rmi -a<br>Monitor disk usage |
| Permission errors | Operation not permitted | Incorrect permissions | Ensure containerd runs as root<br>Check file and socket permissions |

## QUICK CHECKLIST

- containerd installed and running
- SystemdCgroup = true
- CRI socket available
- kubelet configured with systemd cgroup driver
- Test container pulled and running
- No errors in logs

**NOTE:** A stable container runtime is the foundation of a stable Kubernetes cluster.

# 5. BOOTSTRAP THE CLUSTER WITH KUBEADM

kubeadm

## 1 KUBEADM, KUBELET, AND KUBECTL

- kubeadm: bootstrap tool for creating Kubernetes clusters.
- kubelet: runs on every node and manages containers.
- kubectl: command-line tool to interact with the cluster.
- Install all three on every node.

```
sudo apt update
sudo apt install -y kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
```

## 2 CONTROL PLANE INITIALIZATION

- Run on the control plane node.
- Initializes all control plane components.
- Generates certificates and keys.
- Creates static Pod manifests.
- Sets up etcd.
- Outputs join command for workers.

```
sudo kubeadm init \
--control-plane-endpoint cp1.lab.local:6443 \
--pod-network-cidr=10.244.0.0/16
```

## 3 POD NETWORK CIDR

- Specify the Pod network CIDR that matches your CNI plugin.
- Common choice: 10.244.0.0/16 (Calico, Flannel).
- Must not overlap with your LAN.
- Example used in kubeadm init:

```
--pod-network-cidr=10.244.0.0/16
```

**NOTE:** Changing this later requires rebuilding the cluster.

## 4 KUBECONFIG

- After init, kubeconfig is created for the admin user on the control plane.
- Copy it to your management machine.
- Allows kubectl to communicate with the cluster.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

**TIP:** Always run kubectl from your management machine or control plane after copying the config.

## 5 WORKER JOIN PROCESS

- Use the join command generated by kubeadm init.
- Distribute the command securely to worker nodes.
- Worker nodes join and register with the control plane.

```
sudo kubeadm join cp1.lab.local:6443 \
--token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash \
sha256:xxxxxexxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

**TIP:** The token has an expiration. Generate a new one if needed (kubeadm token create --print-join-command).

## 6 BOOTSTRAP TOKENS

- Tokens authenticate workers during join.
- Time-limited for security.
- Can be created, listed, and deleted.
- Default token created during kubeadm init.

```
kubeadm token list
kubeadm token create --print-join-command
kubeadm token delete <token-id>
```

**WARNING:** Do not share tokens publicly. Regenerate if leaked.

## 7 CLUSTER VALIDATION

- Verify all control plane components are running.
- Ensure nodes are Ready.
- Confirm cluster DNS and networking after CNI is installed.

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl get componentstatuses
kubectl cluster-info
kubectl get cs
```

**NOTE:** Install your CNI plugin now so Pods can communicate (e.g., Calico, Flannel, Cilium).

## 8 UNDERSTANDING WHAT KUBEADM ACTUALLY CREATES

- kubeadm sets up the control plane components as static Pods.
- It also creates essential certificates, configs, and RBAC.
- These are the building blocks of your cluster.

| COMPONENT | WHAT KUBEADM CREATES |
|---|---|
| etcd | Local etcd (static Pod) for cluster datastore |
| kube-apiserver | API server (static Pod) – cluster entry point |
| kube-controller-manager | Controllers (static Pod) – node, endpoint, etc. |
| kube-scheduler | Scheduler (static Pod) – workload placement |
| CoreDNS | DNS addon (after CNI) – service discovery |
| kube-proxy | Network proxy on each node (DaemonSet) |
| RBAC | Cluster roles, bindings, and permissions |
| Certificates | CA, admin, kubelet, and component certs |
| kubeconfig | admin.conf for cluster access |
| Cluster State | Initial cluster state in etcd |

## CLUSTER BOOTSTRAP FLOW

Install Prerequisites → Run kubeadm init (Control Plane) → Copy kubeconfig to Admin Machine → Distribute Join Command to Workers → Workers Join the Cluster → Validate Cluster and Install CNI

## SUCCESS CRITERIA

- All control plane Pods are Running
- All nodes are Ready
- Pod networking is working
- You can deploy a test Pod
- Cluster DNS resolves names

# 6. KUBERNETES NETWORKING

## 1 POD-TO-POD COMMUNICATION

- Pods can communicate with each other across nodes.
- No NAT between pods.
- Uses Pod IP addresses.
- Flat network model.

```
Pod A (10.244.1.10)
        ↓
Pod B (10.244.2.15)
```

## 2 POD-TO-SERVICE COMMUNICATION

- Pods access Services using virtual IP and DNS.
- kube-proxy handles Service IP routing.
- ClusterIP is only reachable inside the cluster.
- Load balancing across backend Pods.

```
Pod A
  ↓
Service (ClusterIP)
  ↓
Backend Pods
```

## 3 NODE NETWORKING

- Nodes provide the network infrastructure for Pods.
- Pods on different nodes communicate through the CNI plugin.
- Node IPs are used for host-level communication.
- Each node runs kube-proxy, kubelet, and CNI agent.

```
Node IP Example:
192.168.50.21
```

## 4 CNI RESPONSIBILITIES

- Assign IP addresses to Pods.
- Enable Pod-to-Pod communication.
- Manage routing between nodes.
- Implement network policies.
- Provide bandwidth and network isolation.
- Integrate with Kubernetes CNI specification.

## 5 INSTALLING A CNI

- Popular CNIs: Calico, Flannel, Cilium, Weave Net.
- Apply the CNI manifest file.
- Verify Pods get IP addresses.
- Ensure nodes are Ready.

```
kubectl apply -f calico.yaml
kubectl get pods -n kube-system
kubectl get nodes
```

**NOTE:** CNI must be installed before creating application Pods.

## 6 POD CIDRS

- Each node hosts a range of Pod IPs.
- Defined by the CNI plugin.
- Common choice: 10.244.0.0/16
- Example allocation per node:

| NODE | POD CIDR |
|---|---|
| Node 1 | 10.244.1.0/24 |
| Node 2 | 10.244.2.0/24 |
| Node 3 | 10.244.3.0/24 |
| Node 4 | 10.244.4.0/24 |

**TIP:** Ensure enough IPs for future scaling.

## 7 SERVICE CIDRS

- Service CIDR is used for ClusterIP Services.
- Default: 10.96.0.0/12
- Allocated from the Kubernetes control plane.
- Examples:

| TYPE | CIDR |
|---|---|
| Service CIDR | 10.96.0.0/12 |

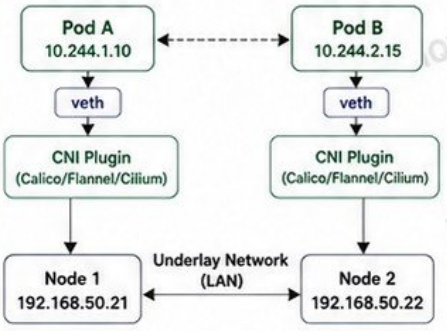
**NOTE:** Do not overlap Service CIDR with your LAN or Pod CIDR.

## 8 KEY NETWORK COMPONENTS

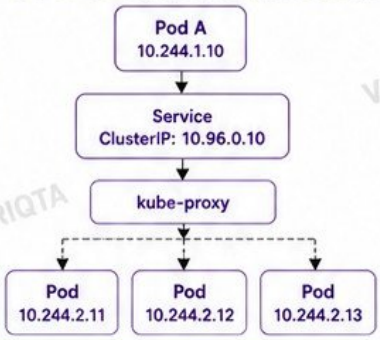
- CNI Plugin – provides Pod network.
- kube-proxy – handles Service networking.
- CoreDNS – provides DNS resolution.
- Network Policies – control traffic between Pods.
- Ingress / LoadBalancer – expose Services externally.

## PACKET-FLOW DIAGRAMS

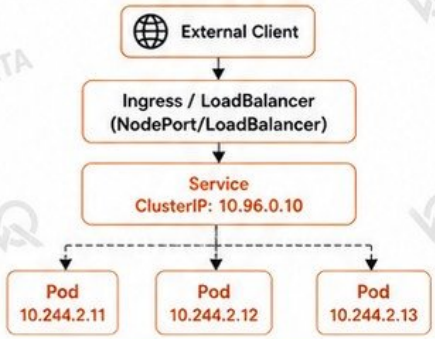
### A. POD-TO-POD COMMUNICATION




### B. POD-TO-SERVICE COMMUNICATION




### C. EXTERNAL ACCESS TO SERVICE




## NETWORKING SUMMARY

- Pods communicate using Pod IPs – no NAT.
- Services provide stable virtual IP and load balancing.
- CNI plugin enables the entire Kubernetes networking model.
- Proper CIDR planning prevents routing and IP conflicts.

## BEST PRACTICES

- Plan CIDRs carefully and document them.
- Verify connectivity after installing CNI.
- Use Network Policies to secure your workloads.
- Monitor network performance and bandwidth usage.

# 7. DNS AND SERVICE DISCOVERY

CoreDNS

## 1 COREDNS

- CoreDNS is the DNS server in Kubernetes.
- Handles DNS resolution for the entire cluster.
- Highly configurable through ConfigMaps.
- Runs as Pods in the kube-system namespace.

```
kubectl get pods -n kube-system
-k8s-app=kube-dns
NAME                       READY   STATUS    RESTARTS
coredns-6779c88dcd-bxck    1/1     Running   0
coredns-6779c88dcd-fgtlj   1/1     Running   0
```

## 2 CLUSTER DNS

- Default cluster DNS IP: `10.96.0.10`
- All Pods use this IP for `dnsPolicy:ClusterFirst`.
- Defined in the kube-dns Service in kube-system namespace.
- Node `/etc/resolv.conf` points to this IP.

**Check CLUSTER DNS:**

```
kubectl get svc -n kube-system kube-dns
NAME        CLUSTER-IP
kube-dns    10.96.0.10
```

## 3 SERVICE NAMES

- Services get a stable DNS name.
- Format: `<service>.<namespace>.svc.cluster.local`
- ClusterIP Service example: `nginx.default.svc.cluster.local`
- Headless Service example: `mysql.db.svc.cluster.local`

**EXAMPLES:**

```
nginx.default.svc.cluster.local
api.backend..svc.cluster.local
redis.cache.svc.cluster.local
```

## 4 NAMESPACE-AWARE RESOLUTION

- DNS queries are namespace-aware.
- If namespace is omitted, the Pod's namespace is used.
- Cross-namespace access requires the full DNS name.
- Search domain example:

```
default.svc.cluster.local
svc.cluster.local
cluster.local
```

## 5 DNS REQUEST FLOW

**1. POD / CLIENT**

Pod sends DNS query for nginx.default.svc.cluster.local to `10.96.0.10`

```
/etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local
options ndots:5
```

→

**2. COREDNS**

CoreDNS receives the query and checks:

- Cache
- ConfigMap
- Kubernetes API

→

**3. KUBERNETES API**

CoreDNS queries the API server to find the Service record (ClusterIP).

→

**4. RESPONSE**

CoreDNS returns the ClusterIP to the Pod.

→

**5. POD RECEIVES IP**

Pod gets the IP and connects to the Service.

Response flows back to the Pod

## 6 TESTING DNS FROM PODS

- Run a temporary Pod with tools.
- Use nslookup, dig, or getent.
- Verify Service and external name resolution.

```
kubectl run -it --rm dnsutils \
--image=registry.k8s.io/e2e-test-images
/jessie-dnsutils:1.3 --restart=Never -- sh

/ # nslookup kubernetes.default \\
Server:         10.96.0.10
Address:    10.96.0.10 kube-dns.kube-system

Name:     kubernetes.default.svc.cluster.local
Address 1: 10.96.0.1 kubernetes.default.svc.
```

## 7 DEBUGGING FAILED NAME RESOLUTION

- Check Pod DNS policy.
- Verify CoreDNS Pod is running.
- Check CoreDNS logs.
- Validate Service exists.
- Test from a different Pod.
- Review NetworkPolicies.
- Check /etc/resolv.conf in Pod.

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
kubectl describe svc <service> -n <namespace>
kubectl exec -it <pod> -- cat /etc/resolv.conf
kubectl exec -it <pod> -- nslookup <service>
```

## 8 COMMON COREDNS PROBLEMS

- CoreDNS Pods CrashLoopBackOff
- Incorrect Corefile syntax
- Upstream DNS unreachable
- High latency or timeouts
- Too many concurrent requests
- Cache issues
- Memory or CPU pressure

**Check Logs:**

```
kubectl logs -n kube-system -l k8s-app=kube-dns
--tail=100 -f
```

## BEST PRACTICES

- Keep CoreDNS updated.
- Monitor CoreDNS metrics.
- Use forwarders for external DNS.
- Limit unnecessary external lookups.
- Review and version-control Corefile.
- Use NetworkPolicies to protect DNS.
- Test DNS as part of cluster health checks.

## COMMON DNS RECORD TYPES IN KUBERNETES

| TYPE | EXAMPLE | DESCRIPTION |
|---|---|---|
| A | nginx.default.svc.cluster.local | Resolves to Service ClusterIP |
| SRV | _http._tcp.nginx.default.svc.cluster.local | Used for headless Services |
| PTR | Reverse lookup | IP to name resolution |
| CNAME | external.example.com | Alias to another name |
| NXDOMAIN | any.unknown.svc.cluster.local | Returned when not found |

## QUICK CHECKLIST

- CoreDNS Pods are Running
- Cluster DNS IP is reachable
- Services resolve by name
- Namespaces resolve correctly
- External DNS works (if configured)

**TIP: DNS is the backbone of Kubernetes. If DNS fails, everything breaks.**

# 8. DEPLOYING THE FIRST REAL WORKLOAD

## FROM YAML TO RUNNING CONTAINERS

kubernetes

### 1. CORE BUILDING BLOCKS

**NAMESPACE**
Virtual cluster segmentation.

**DEPLOYMENT**
Manages ReplicaSets and updates Pods.

**REPLICASET**
Ensures the desired number of Pod replicas.

**PODS**
Smallest deployable units that run containers.

**SERVICE**
Stable network endpoint for Pods.

**CONFIGMAP**
Stores non-sensitive configuration data.

**SECRET**
Stores sensitive information securely.

### 2. RESOURCE REQUESTS AND LIMITS

**REQUESTS**

- Minimum resources guaranteed to the container.
- Used for scheduling decisions.

**LIMITS**

- Maximum resources a container can use.
- Prevents one container from consuming everything.

**EXAMPLE**

```
requests:
  cpu: "100m"     memory: "128Mi"
limits:
  cpu: "500m"     memory: "256Mi"
```

### 4. WORKLOAD LIFECYCLE: FROM YAML TO RUNNING CONTAINERS

**1. WRITE YAML**
Define your Kubernetes objects.

**2. APPLY YAML**
kubectl apply -f workload.yaml creates the objects.

**3. KUBERNETES RECONCILIATION**
API Server stores objects in etcd. Controllers ensure desired state.

**4. POD CREATION**
ReplicaSet creates Pods. Kubelet on nodes starts containers.

**5. RUNNING CONTAINERS**
Your application is now running and reachable via Service.

### WHAT EACH COMPONENT DOES

| COMPONENT | ROLE IN THE WORKLOAD |
|---|---|
| Namespace | Provides isolation for resources. |
| Deployment | Manages updates, rollbacks, and desired replicas. |
| ReplicaSet | Ensures the correct number of Pods are running. |
| Pods | Run your containerized application. |
| Service | Provides a stable IP and DNS name to access Pods. |
| ConfigMap | Injects configuration into Pods. |
| Secret | Injects sensitive data into Pods securely. |
| Resources | Guarantee performance and protect the cluster. |

### 5. TESTING YOUR WORKLOAD

- Check objects:
  `kubectl get all -n demo-app`
- Check Pods:
  `kubectl get pods -n demo-app -o wide`
- Access the app:

```
kubectl port-forward svc/demo-service 8080:80 -n demo-app
# Visit http://localhost:8080
```

- Describe resources:
  `kubectl describe pod <pod-name> -n demo-app`

### 6. COMMON TROUBLESHOOTING

- Pods not starting? Check events:
  `kubectl describe pod <pod-name> -n demo-app`
- Image pull errors? Check image and registry access.
- CrashLoopBackOff? Check logs:
  `kubectl logs <pod-name> -n demo-app --previous`
- Service not reachable? Verify selector and endpoints:
  `kubectl get endpoints -n demo-app`

### 7. KEY TAKEAWAY

You define the desired state in YAML. Kubernetes makes it real by continuously reconciling that state and keeping your workload healthy.

### QUICK CHECKLIST

- Namespace created
- Deployment running
- Pods running
- Service reachable
- ConfigMap mounted
- Secrets working
- Resources set

### 3. COMPLETE WORKLOAD YAML EXAMPLE

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: demo-app
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: demo-app
data:
  APP_ENV: production
  APP_MESSAGE: "Hello from Kubernetes!"
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: demo-app
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQxMjM=  # base64
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
  namespace: demo-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      containers:
      - name: demo-container
        image: nginx:1.25
        ports:
        - containerPort: 80
        env:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_ENV
        - name: APP_MESSAGE
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_MESSAGE
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: DB_PASSWORD
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "256Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: demo-service
  namespace: demo-app
spec:
  type: ClusterIP
  port: 80
```

# 9. INGRESS AND HOME NETWORK ACCESS

kubernetes

## EXPOSING APPLICATIONS TO YOUR LAN

### 1. SERVICE TYPES EXPLAINED

**CLUSTERIP**

- Default service
- Only reachable inside the cluster
- Internal communication only

IP: 10.x.x.x

**NODEPORT**

- Exposes service on each node's IP at a static port
- Reachable from your LAN using <NodeIP>:<Port>

Port Range: 30000-32767

**LOADBALANCER**

- Provisions an external load balancer (Cloud)
- Routes traffic to your service

External IP Assigned

**INGRESS**

- HTTP/HTTPS rules for multiple services
- Host/path based routing
- Works with Ingress Controller

Layer 7 (HTTP) Routing

**INGRESS CONTROLLER**

- Watches Ingress resources
- Implements the rules
- Examples: NGINX, Traefik, HAProxy

Runs as Pods

### 2. METALLB (BARE-METAL LOAD BALANCER)

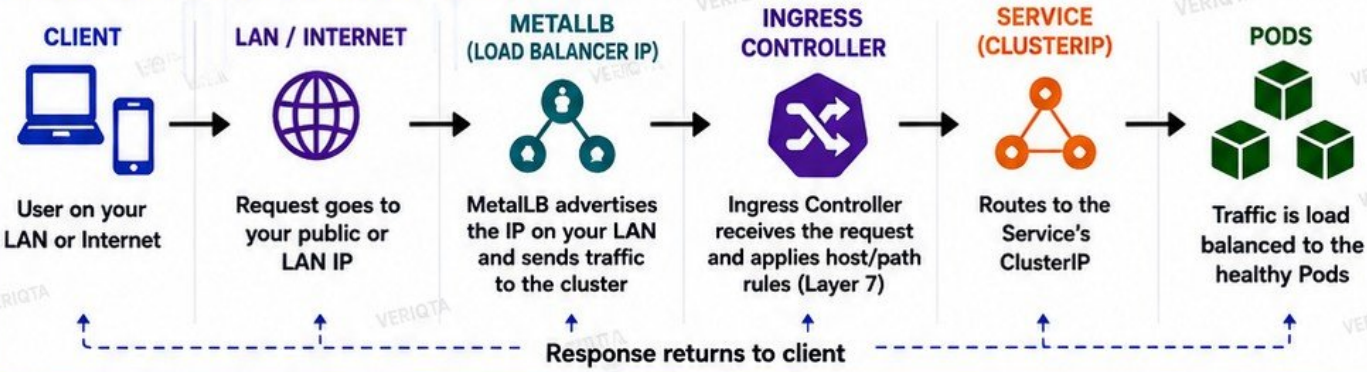
METALLB

- Provides LoadBalancer IPs on bare-metal Kubernetes clusters.
- Acts like a cloud load balancer using your LAN IPs.
- Works with ARP mode (L2) or BGP mode.
- Pairs with Ingress Controller for full production-like access.

### 3. EXPOSING APPLICATIONS TO YOUR LAN

- Use Ingress + Ingress Controller + MetalLB.
- Assign a LAN IP to the LoadBalancer service.
- Access your apps using http://<LAN-IP> or https://<LAN-IP> from any device.
- Supports multiple hostnames and paths.

### 4. REQUEST FLOW DIAGRAM (CLIENT TO POD)




### 5. QUICK COMPARISON

| TYPE | EXPOSED TO LAN | USES PORT | LAYER | BEST FOR | REQUIRES |
|---|---|---|---|---|---|
| ClusterIP | No | No | Layer 4 | Internal communication | None |
| NodePort | Yes | Yes (NodePort) | Layer 4 | Simple access / Testing | Node IP + Port |
| LoadBalancer | Yes | No | Layer 4 | Cloud / External access | Cloud LB or MetalLB |
| Ingress | Yes | No | Layer 7 | HTTP/HTTPS routing | Ingress Controller |
| Ingress Controller | Yes | No | Layer 7 | Implements Ingress rules | Runs In Cluster |

VERIQTA

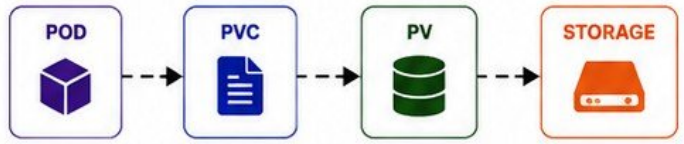
PAGE 10

# 10. PERSISTENT STORAGE

**STORE DATA. KEEP DATA. PROTECT DATA.**

kubernetes

## 1. KUBERNETES STORAGE MODEL




- **POD** — Pod requests storage via PersistentVolumeClaim (PVC)
- **PVC** — PVC is a request for storage with size, access mode and StorageClass
- **PV** — PersistentVolume (PV) is the real piece of storage in the cluster that satisfies the claim
- **STORAGE** — Storage can be on local disk, NFS, cloud disk, or any backend system

## 2. KEY OBJECTS EXPLAINED

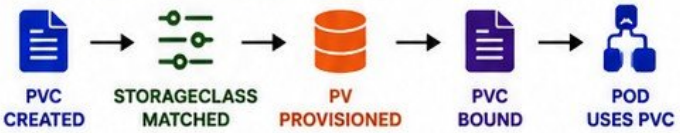
**PERSISTENTVOLUME (PV)**
A cluster resource that represents a piece of storage in the cluster. Created by admin or dynamically.

**PERSISTENTVOLUMECLAIM (PVC)**
A user request for storage. Pods use PVCs, not PVs directly.

**STORAGECLASS**
Defines how storage is provisioned (type, performance, reclaim policy, parameters).

**POD**
Consumes storage by mounting a PVC as a volume.

## 3. DYNAMIC PROVISIONING FLOW




- PVC is created and references a StorageClass.
- Kubernetes finds the matching StorageClass.
- A PV is automatically created by the provisioner.
- PVC binds to the PV.
- Pod mounts the PVC and starts using the storage.

**BENEFITS**
- No manual PV creation
- Faster and simpler
- Scales automatically
- Consistent storage setup

## 4. LOCAL STORAGE LIMITATIONS

- Data tied to a single node.
- If the node fails, data is lost.
- Not portable across nodes.
- No automatic replication.
- Not suitable for High Availability.
- Good for dev/test, not for production.

**USE LOCAL STORAGE ONLY FOR:**

Dev Environments · Testing · Caching · CI/CD Artifacts

## 3. NFS OR DISTRIBUTED STORAGE OPTIONS

**NFS (NETWORK FILE SYSTEM)**

- Simple to set up
- Works with many workloads
- Single point of failure (unless HA)
- Good for shared file storage
- Example: NFS Server on LAN

**DISTRIBUTED STORAGE**

- High availability
- Replicated across nodes
- Better performance & durability
- Examples: Longhorn, Ceph, Rook, OpenEBS, Portworx

## 6. WHAT HAPPENS TO DATA WHEN A POD DIES?

| SCENARIO | WHAT HAPPENS? |
|---|---|
| Pod uses emptyDir (volume) | Data is **LOST** when Pod is deleted (emptyDir is tied to Pod lifecycle). |
| Pod uses PVC (Persistent Storage) | Data is **SAFE** and **PERSISTS** even if Pod is deleted or rescheduled. |
| Node with local storage fails | Data is **LOST**, because storage is tied to that node. |
| Node with NFS or distributed storage fails | Data is **SAFE**, because storage exists outside the node. |

## 7. EXAMPLE: POD USING PVC (NGINX WEB APP)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          volumeMounts:
            - name: web-data
              mountPath: /usr/share/nginx/html
  volumes:
    - name: web-data
      persistentVolumeClaim:
        claimName: web-pvc
```

**1. PVC**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  accessModes: ["ReadWriteMany"]
resources:
  requests:
    storage: 5Gi
  storageClassName: nfs-sc
```

**2. STORAGECLASS**

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-sc
provisioner: nfs.csi.k8s.io
paramtters:
  server: 192.168.50.10
  share: /export/k8s
reclaimPolicy: Retain
volumeBindingMode: Immediate
```

**3. PV (DYNAMIC)**

Automatically created by the provisioner and bound to PVC.

**4. POD RUNNING**

Nginx Pods mount the PVC and serve content from persistent storage.

# 11. DEPLOY A STATEFUL APPLICATION

kubernetes

**STATEFUL WORKLOADS. STABLE IDENTITY. PERSISTENT DATA.**

## 1. KEY CONCEPTS

**STATEFULSET**
Manages the deployment and scaling of a set of Pods, providing guarantees about ordering and identity.

**HEADLESS SERVICE**
Provides stable network identity for Pods. (clusterIP: None)

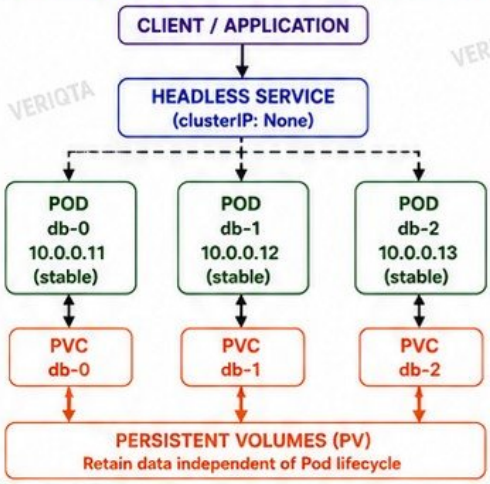
**STABLE POD IDENTITY**
Pods get a stable hostname and DNS name that never changes.

**PERSISTENT VOLUMES**
Each Pod gets its own PersistentVolume via PersistentVolumeClaim (PVC).

**DATABASE DEPLOYMENT**
Ideal for databases like MySQL, PostgreSQL, MongoDB, Redis, etc.

**POD RESCHEDULING**
If a Pod fails, a new Pod with the same identity is created on another node and reattaches the same storage.

## 2. STATEFULSET + HEADLESS SERVICE ARCHITECTURE




- Stable network identity
- Stable storage identity
- Ordered deployment and scaling
- Automatic failover and rescheduling
- Data persists even if Pod or node dies

## 3. HOW IT WORKS

1. You define a StatefulSet and a Headless Service.
2. Kubernetes creates Pods with stable identities (db-0, db-1, db-2).
3. Each Pod gets its own PVC and mounts the PV.
4. If a Pod fails, it is rescheduled with the same identity and PVC.

**BENEFITS**

- Stable identity and DNS
- Ordered deployment and scaling
- Persistent storage per Pod
- Automatic recovery
- Data survives Pod restarts, rescheduling, and node failures

## 4. EXAMPLE: DATABASE STATEFULSET (MYSQL)

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql-headless
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: password
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: fast-local
      resources:
        requests:
          storage: 10Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
  - name: mysql
    port: 3306
    targetPort: 3306
```

## 5. POD IDENTITY AND DNS

| POD NAMES | DNS NAMES (via Headless Service) | POD IPs |
|---|---|---|
| mysql-0 | mysql-0.mysql-headless.default.svc.cluster.local | 10.0.0.11 |
| mysql-1 | mysql-1.mysql-headless.default.svc.cluster.local | 10.0.0.12 |
| mysql-2 | mysql-2.mysql-headless.default.svc.cluster.local | 10.0.0.13 |

Stable DNS names do not change even if Pods are rescheduled.

## 6. DATA PERSISTENCE TESTING

1. Write data to the database.
2. Delete the Pod (simulate failure).
3. Kubernetes creates a new Pod with the same identity.
4. New Pod mounts the same PVC.
5. Data is still available.

**TEST COMMANDS EXAMPLE**

```
# Write some data
echo "test" > /var/lib/mysql/test.txt

# Delete Pod
kubectl delete pod mysql-0

# Wait for recreation
kubectl get pods -w

# Check data
echo "test" should still exist.
```

## 7. STATEFULSET vs DEPLOYMENT

| FEATURE | STATEFULSET | DEPLOYMENT |
|---|---|---|
| Pod Identity | Stable (pod-0, pod-1, ...) | Random (no stable identity) |
| Storage | Persistent (one PVC per Pod) | Ephemeral (shared or none) |
| Deployment Order | Ordered (0 → 1 → 2) | Unordered |
| Scaling | Ordered (one by one) | Parallel |
| Use Cases | Databases, Message Queues, Stateful Apps | Web Apps, APIs, Stateless Apps |

## 8. SUMMARY

- Use StatefulSet for stateful workloads.
- Use Headless Service for stable network identity.
- Use PVCs for persistent storage.
- Data survives Pod restarts, rescheduling, and node failures.
- Perfect for production databases and critical stateful apps.

VERIQTA

PAGE 12

# 12. SCHEDULING AND RESOURCE MANAGEMENT

## 1 SCHEDULER DECISIONS

- Watches for unscheduled Pods
- Filters nodes that do not meet requirements
- Scores remaining nodes
- Binds Pod to the best node
- Considers: resources, policies, constraints

## REQUESTS AND LIMITS

- Requests: Minimum resources guaranteed for the Pod
- Limits: Maximum resources the Pod can use
- Scheduler uses requests, not limits, for placement
- Prevents overcommitment and resource starvation

## NODE SELECTORS

- Constrain Pods to nodes with specific labels
- Uses nodeSelector field
- Simple hard requirement
- Example: disktype=ssd
- Pod runs only on matching nodes

## AFFINITY & ANTI-AFFINITY

- Affinity: Prefer or require Pods on/near certain nodes or other Pods
- Anti-affinity: Avoid placing Pods on/near certain nodes or other Pods
- Improves availability, HA, and performance

## TAINTS AND TOLERATIONS

- Taints repel Pods from nodes
- Tolerations allow Pods to be scheduled on tainted nodes
- Used for dedicated or special-purpose nodes
- Example: node-role.kubernetes.io/gpu:NoSchedule

## POD TOPOLOGY

- TopologySpreadConstraints distribute Pods across zones, nodes, or regions
- Improves resilience and workload balance
- Uses topologySpreadConstraints field

## RESOURCE PRESSURE

- Node conditions: MemoryPressure, DiskPressure, PIDPressure
- Affected nodes may reject new Pods
- Eviction may occur if usage exceeds thresholds
- Monitor with Metrics Server and Prometheus

## SIMULATING PRODUCTION PLACEMENT POLICIES

- Combine selectors, affinity, taints, and topology rules
- Enforce HA and isolation
- Test failure scenarios
- Validate scheduling behavior under load
- Achieve predictable, production-grade placement

## HOW THE SCHEDULER PLACES A POD

1. POD CREATED (UNSCHEDULED) → 2. FILTER NODES (REMOVE INELIGIBLE) → 3. SCORE NODES (RANK REMAINING) → 4. BIND TO BEST NODE

## EXAMPLE PLACEMENT POLICY

A Pod with the following rules:

- Requests: 500m CPU, 1Gi Memory
- nodeSelector: disktype=ssd
- Anti-affinity: Avoid same app on same node
- topologySpreadConstraints: Spread across zones
- Tolerates taint: node-role=app:NoSchedule

The scheduler will:

1. Find nodes labeled disktype=ssd
2. Exclude nodes where the same app is running
3. Prefer nodes in different zones
4. Ensure sufficient free resources
5. Check taints and tolerations
6. Place Pod on the best matching node

## EXAMPLE CLUSTER VIEW

| ZONE A | | PODS | ZONE B | | ZONE C | |
|---|---|---|---|---|---|---|
| NODE A1 | NODE A2 | APP-1 / APP-2 / APP-3 | NODE B1 | NODE B2 | NODE C1 | NODE C2 |
| disktype=ssd<br>zone=us-east-1a<br>cpu: 60%<br>mem: 55% | disktype=ssd<br>zone=us-east-1a<br>cpu: 35%<br>mem: 40% | zone=1a / zone=1b / zone=1c | disktype=ssd<br>zone=us-east-1b<br>cpu: 45%<br>mem: 50% | disktype=ssd<br>zone=us-east-1b<br>cpu: 30%<br>mem: 35% | disktype=ssd<br>zone=us-east-1c<br>cpu: 40%<br>mem: 45% | disktype=ssd<br>zone=us-east-1c<br>cpu: 28%<br>mem: 32% |

TOPOLOGY SPREAD ENSURES PODS ARE DISTRIBUTED ACROSS ZONES

## SCHEDULING BEST PRACTICES

**SET REQUESTS AND LIMITS** — Right-size workloads for efficient scheduling

**USE LABELS CONSISTENTLY** — Clear labels enable predictable placement

**DISTRIBUTE ACROSS ZONES** — Improve HA and fault tolerance

**USE TAINTS FOR DEDICATED NODES** — Isolate workloads and protect critical nodes

**MONITOR RESOURCES** — Detect pressure early and avoid evictions

**TEST POLICIES IN YOUR LAB** — Validate behavior before production rollout

# 13. HEALTH CHECKS AND SELF-HEALING

## KUBERNETES PROBES

- **Liveness probe:** checks if the container is healthy. If it fails, the Pod is restarted.
- **Readiness probe:** checks if the container is ready to serve traffic. If it fails, the Pod is removed from Service.
- **Startup probe:** checks if the container has started successfully.

## PROBE TYPES

| Probe | Purpose | When Used |
|---|---|---|
| Liveness | Detect crashes | Keep Pod healthy |
| Readiness | Detect readiness | Control traffic |
| Startup | Detect startup | Slow start apps |

## POD LIFECYCLE

Pending → Running → Failed / Succeeded

**Goal:** Keep workloads healthy and self-healing.

## POD RESTART BEHAVIOR

- Kubelet restarts containers when liveness probe fails.
- **Restart policy:** Always (default), OnFailure, Never.
- Pod goes back to Running after successful restart.

## REPLICA REPLACEMENT

- Deployment or ReplicaSet creates new Pod when one fails.
- Ensures desired number of replicas is always available.
- Failed Pods are replaced automatically.

## NODE FAILURE

- Pods on the failed node become Unavailable.
- If the Deployment has more replicas, new Pods are scheduled on healthy nodes.
- Kubernetes reschedules work automatically.

## EVICTION

- Node under resource pressure or disk pressure may evict Pods.
- Eviction is triggered by Kubelet.
- Pods are moved to other healthy nodes.

## INTENTIONAL BREAKING WORKLOADS

1. Scale a Deployment to 0 replicas, then back to 3.
2. Kill a container process inside a Pod to trigger a liveness failure.
3. Change a readiness probe to return failure and observe traffic removal.
4. Drain a node (kubectl drain) and watch Pods move to other nodes.
5. Trigger resource pressure to see Pods evicted and rescheduled.

## SELF-HEALING FLOW

Health Probe Fails → Kubelet Takes Action (Restart Pod) → ReplicaSet / Deployment Creates New Pod → Workload Recovers Automatically

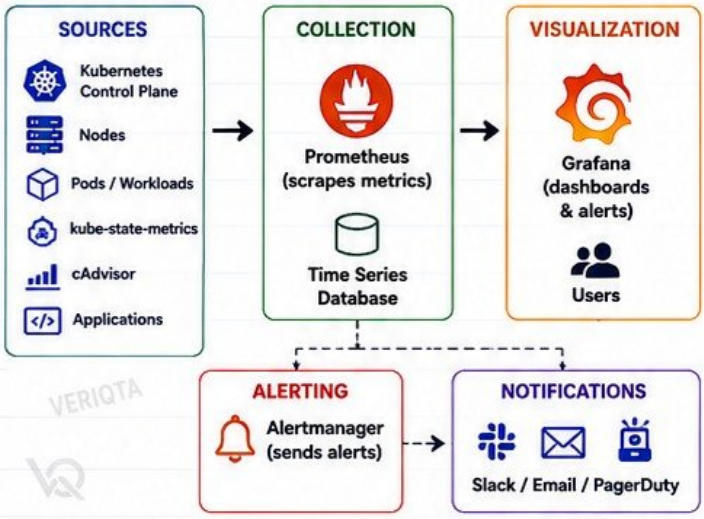
## BEST PRACTICES

- ✓ Use liveness probes for app health
- ✓ Use readiness probes for traffic control
- ✓ Set realistic probe timings
- ✓ Test failure and recovery scenarios

# 14. OBSERVABILITY WITH PROMETHEUS AND GRAFANA

METRICS ARCHITECTURE | PROMETHEUS DEPLOYMENT | GRAFANA DEPLOYMENT | NODE METRICS | POD METRICS | CONTROL-PLANE METRICS | BASIC DASHBOARDS | DETECTING CPU, MEMORY, AND WORKLOAD PROBLEMS

## 1. METRICS ARCHITECTURE




## 2. PROMETHEUS DEPLOYMENT (IN KUBERNETES)

- Deploy Prometheus using helm or manifests
- Components: prometheus server, config, rules
- Service: ClusterIP
- Storage: PersistentVolume (for metrics retention)
- Scrape interval: 15s – 30s
- Retention: 15d (adjust as needed)
- Verify: /targets and /graph in Prometheus UI

## 3. GRAFANA DEPLOYMENT

- Deploy Grafana using helm or manifests
- Service: ClusterIP or LoadBalancer/NodePort
- Persist dashboards in PersistentVolume (optional)
- Add Prometheus as a data source
- Organize dashboards by folders
- Enable alerts and notifications

## 4. WHAT WE MONITOR

| TARGET | WHAT WE GET |
|---|---|
| Nodes (cAdvisor) | CPU, Memory, Disk, Filesystem, Network, Load |
| Pods / Containers | CPU, Memory, Restart, Network I/O, Storage I/O |
| Kubernetes Control Plane | API Server, Scheduler, Controller Manager, etcd performance and availability |
| kube-state-metrics | Deployments, Replicasets, Pods, Nodes, Jobs, CronJobs, Services, PVCs, and more |
| Applications | Custom metrics (requests, latency, errors, etc.) |

## 5. KEY METRICS EXAMPLES

| METRIC | PURPOSE |
|---|---|
| node_cpu_usage_seconds_total | CPU usage per node |
| node_memory_MemAvailable_bytes | Available memory |
| container_cpu_usage_seconds_total | CPU usage per container |
| container_memory_working_set_bytes | Memory used by container |
| kube_pod_status_phase | Pod phase (Running, Pending, etc.) |
| up | Target health (1 = up, 0 = down) |
| apiserver_request_duration_seconds | API server performance |
| etcd_server_has_leader | etcd leader election status |

## 6. BASIC DASHBOARDS YOU SHOULD HAVE

**NODE OVERVIEW**
- CPU usage
- Memory usage
- Disk usage
- Load average
- Network I/O

**POD / WORKLOAD OVERVIEW**
- Pod count
- CPU usage
- Memory usage
- Restarts
- Status

**KUBERNETES CLUSTER OVERVIEW**
- Nodes
- Pods
- Deployments
- Namespaces
- PVCs

**CONTROL PLANE OVERVIEW**
- API server latency
- Scheduler latency
- Controller manager
- etcd health
- CoreDNS

**APPLICATION DASHBOARD**
- Requests per second
- Error rate
- Latency (p95/p99)
- Success rate
- Saturation

## 7. DETECTING CPU, MEMORY, AND WORKLOAD PROBLEMS

**HIGH CPU**
- CPU > 80% for 5m
- Check top pods/containers
- Investigate spikes
- Scale or optimize workloads

**HIGH MEMORY**
- Memory > 80% for 5m
- Check memory usage by pods
- Look for memory leaks
- Increase limits or optimize

**WORKLOAD ISSUES**
- Pod restarts increasing
- Pending pods (insufficient resources)
- CrashLoopBackOff
- High latency or error rate

VERIQTA

PAGE 15

# 15. CENTRALIZED LOGGING

## 1. LOG SOURCES IN KUBERNETES

### A. CONTAINER LOGS
- stdout and stderr from containers
- /var/log/containers/ on each node

### B. NODE-LEVEL LOGS
- kubelet logs
- system logs
- container runtime logs
- OS / kernel logs

### C. APPLICATION LOGS
- App-specific logs inside containers
- Custom log files
- Business events

## 2. LOG COLLECTION ARCHITECTURE

**LOG SOURCES** (Pods, Nodes, Applications) → **COLLECTOR** (Fluent Bit) Collects, parses, and enriches logs → **CENTRAL STORAGE** Stores logs centrally → **VISUALIZATION & ANALYSIS** Search, visualize, alert, and monitor

## 3. FLUENT BIT (OR EQUIVALENT COLLECTOR)

fluentbit

- Lightweight and high performance
- Runs as DaemonSet on all nodes
- Collects logs from files, containers, and system sources
- Parses, filters, and enriches logs
- Forwards to central storage
- Low resource usage
- Supports many outputs

## 4. CENTRAL LOG STORAGE OPTIONS

### ELASTICSEARCH
- Powerful search
- Scalable
- Great for observability

### LOKI
- Designed for logs
- Cost-effective
- Works great with Grafana

### OPENSEARCH
- Fork of Elasticsearch
- Fully open source
- Log analytics and monitoring

### S3 / MINIO
- Object storage
- Long-term retention
- Cost-effective archival

## 5. FOLLOWING LOG ACROSS REPLICAS

**MULTIPLE REPLICAS** Same app running across pods → **COLLECTOR** Fluent Bit collects logs from all pods → **CENTRAL STORAGE** Logs stored with pod metadata → **SEARCH & FILTER** Search by app name, pod name, namespace, label, or request ID

### USEFUL SEARCH FIELDS
- namespace
- pod
- container
- label
- host
- log
- timestamp
- level
- request_id
- trace_id

## 6. WHAT TO LOG

- Application logs (INFO, WARN, ERROR)
- Access logs (HTTP requests)
- Audit logs (security events)
- System logs (OS, kernel)
- Kubernetes control plane logs
- Database / cache logs
- Background job logs
- Any custom business events

## 7. TROUBLESHOOTING MISSING LOGS

**POD CRASHING TOO EARLY**
- Logs may be lost if container exits before collection.

**LOG PATH INCORRECT**
- Ensure collector is reading the correct log path.

**PERMISSIONS ISSUE**
- Collector must have read access to log files.

**BUFFER OVERFLOW**
- Increase buffer size or flush interval.

**NETWORK ISSUES**
- Collector cannot reach central storage.

**STORAGE FULL**
- Disk full on node or storage backend.

**FILTERS DROPPING LOGS**
- Check parsing / filtering rules.

**TIME SYNC ISSUE**
- Incorrect timestamps can hide logs.

## 8. BEST PRACTICES

- Collect everything, filter later
- Use structured logging (JSON)
- Include request IDs for tracing
- Set log retention policies
- Monitor your logging pipeline
- Alert on logging failures
- Test log search daily
- Document your logging setup

# 16. SECURITY HARDENING

**SECURE YOUR CLUSTER. PROTECT YOUR WORKLOADS. REDUCE RISK.**

## 1. RBAC

- Use Role and RoleBinding
- Use ClusterRole and ClusterRoleBinding
- Grant only required permissions
- Avoid using cluster-admin
- Review access regularly

**PRINCIPLE: LEAST PRIVILEGE FOR ALL USERS AND SERVICES**

## 2. SERVICEACCOUNTS

- Use dedicated ServiceAccounts for each application
- Do not use the default ServiceAccount
- Disable automatic mounting of tokens
- Bind only necessary roles

**ONE APPLICATION = ONE SERVICEACCOUNT**

## 3. LEAST PRIVILEGE

- Grant minimum permissions required
- Avoid wildcard permissions
- Restrict access to namespaces
- Use resource and verb restrictions
- Regular permission reviews

**GIVE JUST ENOUGH ACCESS TO GET THE JOB DONE**

## 4. SECRETS HANDLING

- Store sensitive data in Secrets
- Enable encryption at rest (etcd encryption provider)
- Use sealed-secrets or external secret store
- Limit secret access with RBAC
- Never expose secrets in logs

**PROTECT SECRETS IN TRANSIT, AT REST, AND IN USE**

## 5. SECURITYCONTEXT

- Define securityContext at Pod and container level
- Drop all capabilities
- Run as non-root
- Set readOnlyRootFilesystem
- Restrict privilege escalation

**APPLY STRONG SECURITY CONTEXT TO EVERY WORKLOAD**

## 6. NON-ROOT CONTAINERS

- Run containers as non-root user
- Set runAsUser and runAsGroup
- Avoid root (UID 0)
- Use distroless or minimal base images
- Verify file permissions

**ASSUME BREACH. LIMIT THE DAMAGE WITH NON-ROOT.**

## 7. NETWORKPOLICY

- Default deny all ingress and egress
- Allow only necessary traffic
- Isolate namespaces
- Control Pod-to-Pod communication
- Use labels for fine-grained rules

**ZERO TRUST NETWORKING INSIDE THE CLUSTER**

## 8. IMAGE SECURITY

- Use trusted base images
- Scan images for vulnerabilities
- Keep images up to date
- Use imagePullPolicy: Always
- Sign images (cosign / Notary)
- Avoid privileged or unnecessary packages

**BUILD SECURE IMAGES. SCAN. VERIFY. TRUST.**

## 9. PROTECTING THE KUBERNETES API

- Enable TLS for API server
- Use strong authentication
- Enable RBAC authorization
- Restrict access to API server (firewall / security groups)
- Rotate certificates regularly
- Audit API server access

**THE API SERVER IS THE BRAIN. SECURE IT AT ALL TIMES.**

## BEST PRACTICES CHECKLIST

- [x] Use RBAC and follow least privilege
- [x] Use dedicated ServiceAccounts
- [x] Disable unnecessary permissions
- [x] Encrypt Secrets at rest
- [x] Scan container images regularly
- [x] Run containers as non-root
- [x] Set proper securityContext
- [x] Apply NetworkPolicies
- [x] Restrict API server access
- [x] Enable audit logging
- [x] Keep Kubernetes and nodes updated
- [x] Monitor and review security posture
- [x] Document and enforce policies

## QUICK SECURITY CHECKS (EXAMPLES)

| | |
|---|---|
| `kubectl auth can-i --list` | Check your effective permissions |
| `kubectl get sa -A` | List ServiceAccounts |
| `kubectl get secrets -A` | List Secrets (ensure restricted access) |
| `kubectl get pods -A -o wide` | Check running Pods and nodes |
| `kubectl get networkpolicies -A` | List NetworkPolicies |
| `kubectl get --raw /healthz` | Check API server health |
| `kubectl logs <pod-name>` | View Pod logs |
| `kubectl describe pod <pod>` | Inspect Pod security context |

VERIQTA

PAGE 17

# 17. BACKUP AND DISASTER RECOVERY

**BACKUP TODAY. RESTORE TOMORROW. STAY PREPARED.**

## 1. WHAT ACTUALLY NEEDS BACKING UP

- etcd data (cluster state)
- Persistent application data
- Kubernetes manifests
- Cluster configuration
- Secrets and credentials
- StorageClass and PV info
- RBAC and security policies
- Custom resources (CRDs)
- DNS and network configs

## 2. ETCD SNAPSHOTS

- etcd stores the entire cluster state
- Take regular snapshots
- Use a consistent snapshot method
- Encrypt and store securely
- Verify snapshot integrity
- Keep multiple versions
- Snapshot + Long-term storage

## 3. PERSISTENT APPLICATION DATA

- Identify all PVs
- Understand storage backend
- Backup databases
- Backup object storage
- Use filesystem or application-level backups
- Test data restore independently

## 4. KUBERNETES MANIFESTS

YAML

- Backup all manifests
- Deployments, Services, Ingress, ConfigMaps, etc.
- Include CRDs and Operators
- Version control (Git)
- Capture cluster add-ons manifests

## 5. CLUSTER CONFIGURATION

- kubeadm config
- kubelet configuration
- CNI configuration
- CoreDNS configuration
- Ingress controller config
- Node configurations
- Certificates (if not auto generated)

## 6. RESTORE WORKFLOW

1. Prepare new or existing infrastructure
2. Install Kubernetes
3. Restore etcd snapshot
4. Restore persistent data
5. Reapply cluster manifests
6. Restore configs and secrets
7. Validate cluster state
8. Test workloads

## 7. TESTING RECOVERY

- Schedule regular restore tests
- Test in isolated environment
- Validate application data
- Validate cluster functionality
- Measure RTO and RPO
- Document results
- Fix issues before disaster happens

## 8. WHY AN UNTESTED BACKUP IS NOT A RECOVERY STRATEGY

- Backups can be corrupt
- Restores can fail
- Missing dependencies
- Wrong assumptions
- Unknown errors
- Disaster is the worst time to discover problems

## BACKUP STRATEGY REFERENCE

| ETCD SNAPSHOTS | PERSISTENT DATA | MANIFESTS | CONFIGURATION | SECRETS & CERTIFICATES | RETENTION POLICY |
|---|---|---|---|---|---|
| Every 6–12 hours<br>Keep 7–30 days<br>Encrypt and securely store off-cluster | Daily (or per change)<br>App-level backup<br>Retain based on business needs | On every change<br>Store in Git<br>Tag important versions | Weekly or after changes<br>Store securely and versioned | Backup securely<br>Rotate regularly<br>Use external secret management | Define retention per data type<br>Follow 3-2-1 rule |

## 3-2-1 BACKUP RULE

**3** — KEEP 3 COPIES OF YOUR DATA
1 Primary + 2 Backups

**2** — USE 2 DIFFERENT TYPES OF MEDIA
(e.g., Disk + Cloud)

**1** — KEEP 1 COPY OFF-SITE OR OFF-CLUSTER
Protect against disaster

**A BACKUP YOU HAVEN'T TESTED IS JUST A GUESS.**
**TEST TODAY. RECOVER TOMORROW.**

# 18. UPGRADES AND CLUSTER MAINTENANCE

**PLAN CAREFULLY. UPGRADE SAFELY. MAINTAIN RELIABLY.**

## 1. KUBERNETES VERSION SKEW

- Control plane can be newer than workers
- Supported skew: N-1 minor version (e.g., 1.29 control plane with 1.28 workers)
- kubectl skew ±1 minor
- Check compatibility before upgrading

## 2. CONTROL-PLANE UPGRADE

- Upgrade one control plane node at a time
- Follow official upgrade order
- etcd is upgraded with the control plane
- Verify cluster health after each step

## 3. WORKER-NODE UPGRADE

- Upgrade worker nodes one at a time
- Drain before upgrade
- Verify node is Ready and workloads running
- Ensure kubelet and kube-proxy versions match cluster

## 4. CORDON

- Mark node as unschedulable
- Prevents new Pods from being placed on the node
- Existing Pods keep running

```
kubectl cordon <node>
```

## 5. DRAIN

- Evict Pods safely
- Respects PodDisruption Budgets
- Waits for Pods to terminate gracefully

```
kubectl drain <node>
--ignore-daemonsets
--delete-emptydir-data
```

## 6. UNCORDON

- Mark node as schedulable again
- New Pods can be placed on the node

```
kubectl uncordon <node>
```

## 7. PODDISRUPTION BUDGET (PDB)

- Limits the number of Pods that can be unavailable
- Protects application availability during maintenance
- Define minAvailable or maxUnavailable

```
apiVersion: policy/v1
kind: PodDisruptionBudget
```

## 8. SAFE MAINTENANCE WORKFLOW

1. Review release notes
2. Check version skew
3. Backup etcd and manifests
4. Cordon node
5. Drain node (respect PDB)
6. Upgrade component
7. Verify health
8. Uncordon node
9. Repeat for remaining nodes

## 9. ROLLBACK CONSIDERATIONS

- Not all upgrades are reversible
- Backup etcd before upgrade
- Keep previous binaries available
- Test rollback procedure before emergency
- Follow documented downgrade steps (if supported)

## 10. BEST PRACTICES

- Upgrade in a non-production environment first
- Monitor cluster after each upgrade step
- Automate upgrades where possible
- Document your upgrade and rollback plan
- Never skip backups

## UPGRADE ORDER REFERENCE

1. CONTROL PLANE NODE 1 → 2. CONTROL PLANE NODE 2 → 3. CONTROL PLANE NODE 3 (IF ANY) → 4. WORKER NODE 1 → 5. WORKER NODE 2 → 6. WORKER NODE N

## PODDISRUPTIONBUDGET EXAMPLE

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-app-pdb
spec:
 minAvailable: 2
 selector:
  matchLabels:
    app: web-app
```

If 3 replicas are running and minAvailable is 2:

- Max 1 Pod can be down at a time
- Ensures high availability during maintenance

## SAFE MAINTENANCE FLOW

PLAN → BACKUP → CORDON → DRAIN → UPGRADE → VERIFY → UNCORDON

**MAINTENANCE WITHOUT A PLAN IS AN OUTAGE WAITING TO HAPPEN.**
**BACKUP. PLAN. TEST. THEN EXECUTE.**

VERIQTA

PAGE 19

# 19. KUBERNETES TROUBLESHOOTING LAB

FIND THE PROBLEM. UNDERSTAND THE CAUSE. RESTORE THE SERVICE.

## 1. PENDING PODS

- Check pod events
- Insufficient resources
- Failed scheduling
- Storage or affinity issues
- Check quotas, limits, taints, and node capacity

```
kubectl describe pod <pod>
```

## 2. CRASHLOOPBACKOFF

- Check container logs
- Application errors
- Misconfiguration
- Failed health probes
- Resource limits
- Dependency issues

```
kubectl logs <pod> -c <container>
kubectl describe pod <pod>
```

## 3. IMAGEPULLBACKOFF

- Wrong image name
- Image not in registry
- Authentication issues
- Registry unreachable
- Image tag incorrect

```
kubectl describe pod <pod>
kubectl get events
```

## 4. FAILED SCHEDULING

- Check events
- Taints and tolerations
- Node selector mismatch
- Insufficient resources
- Pod affinity rules

```
kubectl describe pod <pod>
kubectl get nodes -o wide
```

## 5. BROKEN DNS

- CoreDNS down
- Network policy blocking
- DNS misconfiguration
- Service discovery issues
- Check /etc/resolv.conf in pods

```
kubectl exec -it <pod> -- nslookup <svc>
kubectl get pods -n kube-system
```

## 6. UNREACHABLE SERVICES

- Endpoints not ready
- Selector mismatch
- Network policy blocking
- Service type issues
- Check kube-proxy

```
kubectl get endpoints <svc>
kubectl describe svc <svc>
```

## 7. CNI FAILURES

- Pods cannot communicate
- CNI plugin down
- IP address exhaustion
- MTU mismatch
- Check CNI logs

```
kubectl get pods -n <cni-namespace>
kubectl logs -n <cni-namespace> -l app=<cni>
```

## 8. STORAGE FAILURES

- PVC not bound
- StorageClass issues
- Disk full
- Permission problems
- CSI driver down

```
kubectl get pvc,pv
kubectl describe pvc <pvc>
```

## 9. NODE NOTREADY

- Kubelet down
- Network issues
- Resource pressure
- Disk pressure
- Check node events

```
kubectl describe node <node>
journalctl -u kubelet -f
```

## 10. INVESTIGATION TOOLKIT

| EVENTS | DESCRIBE | LOGS | EXEC | NODE-LEVEL |
|---|---|---|---|---|
| See what happened | Deep details about objects | View logs from containers | Run commands inside a pod | Check services, resources, and logs |
| `kubectl get events --sort-by=.lastTimestamp` | `kubectl describe <pod\|svc\|node\|pvc>` | `kubectl logs <pod> -c <container> -f` | `kubectl exec -it <pod> -- bash` | `systemctl status kubelet` `journalctl -u kubelet -f` |

## TROUBLESHOOTING WORKFLOW

1. IDENTIFY THE SYMPTOM
2. GATHER INFORMATION (Events, Logs, Describe)
3. FIND THE ROOT CAUSE
4. APPLY THE FIX
5. VERIFY THE RESOLUTION
6. DOCUMENT THE LESSON

## PRO TIPS

- Always start with events
- Reproduce the issue
- Change one thing at a time
- Verify the fix
- Document what you learned

VERIQTA

PAGE 20

# 20. FINAL PRODUCTION-STYLE HOME LAB

BUILD IT. BREAK IT. UNDERSTAND IT. OPERATE IT LIKE PRODUCTION.

## BUILD THE COMPLETE ENVIRONMENT

1. MULTI-NODE KUBERNETES CLUSTER
2. CNI NETWORKING
3. METALLB
4. INGRESS CONTROLLER
5. DNS (COREDNS)
6. PERSISTENT STORAGE
7. STATEFUL WORKLOAD
8. STATELESS APPLICATION
9. CONFIGMAPS AND SECRETS
10. RBAC
11. NETWORKPOLICY
12. PROMETHEUS
13. GRAFANA
14. CENTRALIZED LOGGING
15. BACKUP STRATEGY
16. FAILURE TESTING

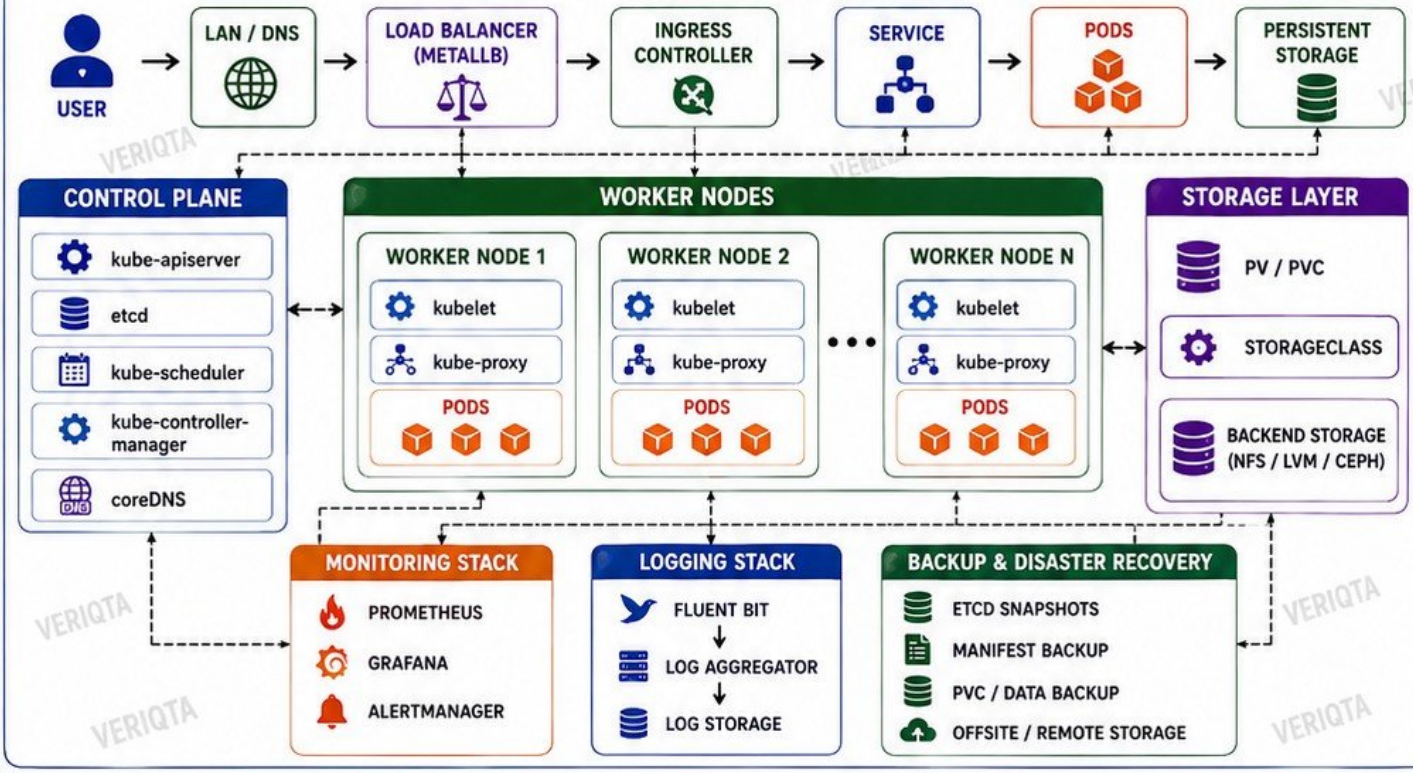
## FINAL CHALLENGE

SIMULATE A PRODUCTION INCIDENT BY TAKING A WORKER NODE OFFLINE.

- OBSERVE WORKLOAD BEHAVIOR
- IDENTIFY WHAT KUBERNETES AUTOMATICALLY RECOVERS
- IDENTIFY WHAT DOES NOT RECOVER
- TROUBLESHOOT THE REMAINING FAILURES
- DOCUMENT THE RECOVERY PROCESS

THE GOAL IS CONFIDENCE, NOT JUST A WORKING LAB.

## COMPLETE HOME LAB ARCHITECTURE




A TRUE PRODUCTION-LIKE LAB IS NOT BUILT ONCE.
IT IS TESTED, BROKEN, RECOVERED, AND IMPROVED CONTINUOUSLY.